

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: CSA14

COURSE NAME: COMPILER DESIGN

COURSE OUTCOME COVERED

CO3: *Analyze syntax-directed translation method using synthesized and inherited attributes to generate a Parser Tree. (BL4)*

Assessment Tool Weightage: 40%

QUESTIONS:5

Mode: In Class

Total Marks:100

Hours : 60 mts

Annexure A

EXPERIMENTS

*I G. Poojitha (192425337) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student: G. Poojitha

Q. No	Understanding of Concepts (25)	Experimental Design (30)	Use of Tools (20)	Analysis of Results (15)	Documentation (10)	Total Score (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	/ 4	
OVERALL RUBRICS VALUE						
	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 20	/ 20	/ 25	/ 20	/ 15	/ 100

Annexure A

EXPERIMENTS

Q1.

You are designing a compiler module that evaluates arithmetic expressions using syntax-directed definitions (SDD). The system must:

- (i) Accept arithmetic expressions with operators +, -, *, / and parentheses
- (ii) Construct a syntax tree and evaluate expressions using synthesized attributes
- (iii) Respect operator precedence and associativity rules
- (iv) Produce the final computed value of the expression

Demonstrate how SDD rules are applied for evaluating expressions and explain the bottom-up evaluation process using annotated parse trees.

Q2.

In a compiler, intermediate code generation often uses postfix expressions for efficient evaluation. The system must:

- (i) Accept a postfix expression as input
- (ii) Use a stack-based approach to evaluate the expression
- (iii) Demonstrate bottom-up computation of intermediate results
- (iv) Display the final evaluated result

Write a C program to evaluate postfix expressions and explain how this process relates to S-attributed definitions in compilers.

Q3.

You are developing a compiler component that uses L-attributed definitions for expression evaluation. The system must:

- (i) Accept arithmetic expressions such as $a + b * c$
- (ii) Use inherited attributes to pass information during parsing
- (iii) Ensure correct operator precedence
- (iv) Perform evaluation using a top-down approach

Write a C program that simulates L-attributed evaluation and explain how inherited attributes are used to propagate values during parsing.

Q4.

A compiler's semantic analyzer must check type compatibility between expressions. The system should:

- (i) Accept two type expressions such as int, float, int*, char*
- (ii) Compare the types for equivalence
- (iii) Handle both basic and pointer types
- (iv) Display whether the types are equivalent or not

Write a C program to perform type equivalence checking and explain how compilers use this mechanism during semantic analysis.

Q5.

You are designing a semantic analyzer for a compiler that detects type errors in arithmetic expressions. The system must:

- (i) Accept operands along with their types
 - (ii) Validate operations such as +, -, , /
 - (iii) *Detect invalid combinations such as int + char*
 - (iv) Report whether the expression is valid or contains a type error
- Write a C program to perform type checking and explain how semantic analysis ensures correctness in program execution.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

Q1.

You are designing a compiler module that evaluates arithmetic expressions using syntax-directed definitions (SDD). The system must:

- (i) Accept arithmetic expressions with operators +, -, *, / and parentheses**
- (ii) Construct a syntax tree and evaluate expressions using synthesized attributes**
- (iii) Respect operator precedence and associativity rules**
- (iv) Produce the final computed value of the expression**

Demonstrate how SDD rules are applied for evaluating expressions and explain the bottom-up evaluation process using annotated parse trees.

Software Tool Used : VS code

Programming Language Used : Python

Code:

```
C: > Users > G POOJITHA > sdd_evaluator.py
1 class Node:
2     def __init__(self, v, l=None, r=None):
3         self.v, self.l, self.r, self.val = v, l, r, None
4
5 def parse(s):
6     out, ops = [], []
7     for t in s.replace("(", " ( ").replace(")", " ) ").split():
8         if t.replace(".", "", 1).isdigit():
9             out.append(t)
10        elif t == "(":
11            ops.append(t)
12        elif t == ")":
13            while ops[-1] != "(": out.append(ops.pop())
14            ops.pop()
15        else:
16            p = 1 if t in "+-" else 2
17            while ops and ops[-1] != "(" and (1 if ops[-1] in "+-" else 2) >= p:
18                out.append(ops.pop())
19            ops.append(t)
20
21    while ops: out.append(ops.pop())
22
23    st = []
24    for t in out:
25        if t.replace(".", "", 1).isdigit():
26            st.append(Node(t))
27        else:
28            r, l = st.pop(), st.pop()
29            st.append(Node(t, l, r))
30    return st[0], out
31
```

```

32 def evaluate(n):
33     if not n.l:
34         n.val = float(n.v)
35     else:
36         a, b = evaluate(n.l), evaluate(n.r)
37         n.val = [{"+": a+b, "-": a-b, "*": a*b, "/": a/b}][n.v]
38     return n.val
39
40 def tree(n, d=0):
41     if n:
42         print("  " * d + f"{n.v}[{n.val}]" )
43         tree(n.l, d+1)
44         tree(n.r, d+1)
45
46 try:
47     exp = input("Enter expression: ")
48     root, postfix = parse(exp)
49     result = evaluate(root)
50
51     print("\nPostfix:", " ".join(postfix))
52     print("\nAnnotated Syntax Tree:")
53     tree(root)
54     print("\nFinal Value:", result)
55
56 except ZeroDivisionError:
57     print("Error: Division by zero")
58 except Exception:
59     print("Error: Invalid expression")

```

Output:

```

Enter expression: 10 + 2 * 3 - (8 / 4)

Postfix: 10 2 3 * + 8 4 / -

Annotated Syntax Tree:
-[14.0]
  +[16.0]
    10[10.0]
    *[6.0]
      2[2.0]
      3[3.0]
    /[2.0]
      8[8.0]
      4[4.0]

Final Value: 14.0
PS C:\Users\G POOJITHA> 

```

Q2.

In a compiler, intermediate code generation often uses postfix expressions for efficient evaluation. The system must:

- (i) Accept a postfix expression as input
- (ii) Use a stack-based approach to evaluate the expression
- (iii) Demonstrate bottom-up computation of intermediate results
- (iv) Display the final evaluated result

Write a C program to evaluate postfix expressions and explain how this process relates to S-attributed definitions in compilers.

Software Tool Used : Online GDB

Code:

```
#include <stdio.h>
#include <ctype.h>

int main() {
    char exp[100], c;
    int stack[100], top = -1, a, b, i;

    printf("Enter postfix expression: ");
    scanf("%s", exp);

    for (i = 0; exp[i]; i++) {
        c = exp[i];

        if (isdigit(c))
            stack[++top] = c - '0';

        else {
            b = stack[top--];
            a = stack[top--];

            switch (c) {
                case '+': stack[++top] = a + b; break;
                case '-': stack[++top] = a - b; break;
                case '*': stack[++top] = a * b; break;
                case '/': stack[++top] = a / b; break;
            }

            printf("%d %c %d = %d\n", a, c, b, stack[top]);
        }
    }

    printf("Final Result = %d\n", stack[top]);
    return 0;
}
```

Output:

```
Enter postfix expression: 23*54*+  
2 * 3 = 6  
5 * 4 = 20  
6 + 20 = 26  
Final Result = 26  
  
...Program finished with exit code 0  
Press ENTER to exit console.
```

Q3.

You are developing a compiler component that uses L-attributed definitions for expression evaluation. The system must:

- (i) Accept arithmetic expressions such as $a + b * c$
- (ii) Use inherited attributes to pass information during parsing
- (iii) Ensure correct operator precedence
- (iv) Perform evaluation using a top-down approach

Write a C program that simulates L-attributed evaluation and explain how inherited attributes are used to propagate values during parsing.

Software Tool Used : Github Codespaces

C q3_l_attributed.c

```
1  #include <stdio.h>
2  #include <ctype.h>
3  char s[100];
4  int i = 0;
5  double val[26] = {0};
6  double E(double inh);
7  double F() {
8      if (s[i] == '(') {
9          i++;
10         double x = E(0);
11         i++;
12         return x;
13     }
14
15     char c = s[i++];
16     return isalpha(c) ? val[c - 'a'] : c - '0';
17 }
18 double T() {
19     double x = F();
20
21     while (s[i] == '*' || s[i] == '/') {
22         char op = s[i++];
23         double y = F();
24         x = (op == '*') ? x * y : x / y;
25     }
26     return x;
27 }
28
29 /* L-attributed top-down evaluation */
30 double E(double inh) {
31     double x = T();
32 }
```



```

33     while (s[i] == '+' || s[i] == '-') {
34         char op = s[i++];
35         double y = T();
36         /* Inherited attribute propagated left-to-right */
37         inh = (op == '+') ? x + y : x - y;
38         x = inh;
39
40         printf("Inherited attribute = %.2f\n", inh);
41     }
42     return x;
43 }
44 int main() {
45     printf("Enter expression (e.g. a+b*c): ");
46     scanf("%s", s);
47     /* Values for variables */
48     val['a' - 'a'] = 2;
49     val['b' - 'a'] = 3;
50     val['c' - 'a'] = 4;
51
52     printf("\nVariable values: a=2, b=3, c=4\n");
53     double result = E(0);
54     if (i != 0 && s[i] != '\0')
55         printf("Invalid expression\n");
56     else
57         printf("\nFinal Result = %.2f\n", result);
58     return 0;
59 }

```

Output:

Enter expression (e.g. a+b*c): a+b*c

Variable values: a=2, b=3, c=4

Inherited attribute = 14.00

Q4.

A compiler's semantic analyzer must check type compatibility between expressions. The system should:

- (i) Accept two type expressions such as int, float, int*, char*
- (ii) Compare the types for equivalence
- (iii) Handle both basic and pointer types
- (iv) Display whether the types are equivalent or not

Write a C program to perform type equivalence checking and explain how compilers use this mechanism during semantic analysis.

Software Tool Used : Cloud Shell Editor

Code:

```
home > gandikotapoojitha2 > C q4_type_equivalence.c > equivalent
1  #include <stdio.h>
2  #include <string.h>
3  int equivalent(char a[], char b[]) {
4      char *p1 = strchr(s: a, c: '*');
5      char *p2 = strchr(s: b, c: '*');
6
7      /* Both basic types */
8      if (!p1 && !p2)
9          return strcmp(s1: a, s2: b) == 0;
10
11     /* Both pointer types */
12     if (p1 && p2)
13         return strncmp(s1: a, s2: b, n: p1 - a) == 0;
14
15     /* One pointer and one basic type */
16     return 0;
17 }
18 int main() {
19     char t1[20], t2[20];
20
21     printf(format: "Enter first type (int/float/int*/char*): ");
22     scanf(format: "%19s", t1);
23
24     printf(format: "Enter second type (int/float/int*/char*): ");
25     scanf(format: "%19s", t2);
26
27     printf(format: "\nType 1: %s\nType 2: %s\n", t1, t2);
28
29     if (equivalent(a: t1, b: t2))
30         printf(format: "Types are EQUIVALENT\n");
31     else
32         printf(format: "Types are NOT EQUIVALENT\n");
33
34     return 0;
35 }
```

Output:

Test Case 1:

```
Enter first type (int/float/int*/char*): int
Enter second type (int/float/int*/char*): int

Type 1: int
Type 2: int
Types are EQUIVALENT
```

Test Case 2:

```
Enter first type (int/float/int*/char*): int
Enter second type (int/float/int*/char*): char

Type 1: int
Type 2: char
Types are NOT EQUIVALENT
```

Q5.

You are designing a semantic analyzer for a compiler that detects type errors in arithmetic expressions. The system must:

- (i) Accept operands along with their types**
- (ii) Validate operations such as +, -, , /**
- (iii) Detect invalid combinations such as int + char**
- (iv) Report whether the expression is valid or contains a type error**

Write a C program to perform type checking and explain how semantic analysis ensures correctness in program execution.

Software Tool Used : Google Colab

Code:

```
%%writefile q5_type_checker.c
#include <stdio.h>
#include <string.h>

int isNumeric(char type[]) {
    return strcmp(type, "int") == 0 ||
           strcmp(type, "float") == 0 ||
           strcmp(type, "char") == 0;
}

int main() {
    char op1[20], op2[20];
    char type1[20], type2[20];
    char op;

    printf("Enter first operand: ");
    scanf("%s", op1);

    printf("Enter type of first operand: ");
    scanf("%s", type1);

    printf("Enter operator (+ - * /): ");
    scanf(" %c", &op);

    printf("Enter second operand: ");
    scanf("%s", op2);
```

```

printf("Enter type of second operand: ");
scanf("%s", type2);

printf("\nExpression: %s(%s) %c %s(%s)\n",
      op1, type1, op, op2, type2);

if (op != '+' && op != '-' && op != '*' && op != '/') {
    printf("Invalid operator!\n");
}
else if (!isNumeric(type1) || !isNumeric(type2)) {
    printf("TYPE ERROR: Incompatible operands\n");
}
else {
    printf("Expression is VALID\n");
    printf("Semantic analysis: No type error detected.\n");
}

return 0;
}

```

Writing q5_type_checker.c

Output:

```

Enter first operand: a
Enter type of first operand: int
Enter operator (+ - * /): +
Enter second operand: p
Enter type of second operand: char

Expression: a(int) + p(char)
Expression is VALID
Semantic analysis: No type error detected.

```